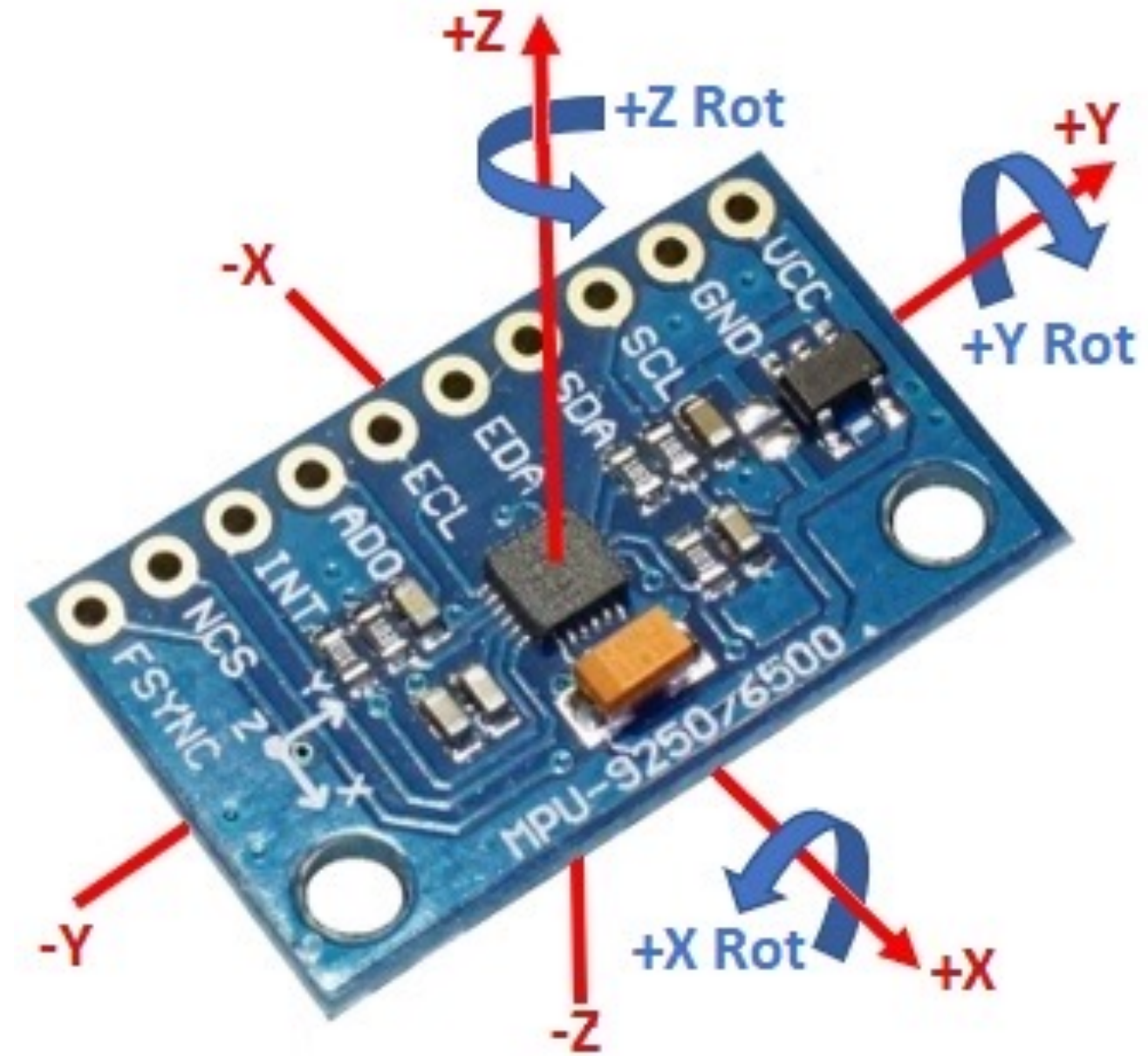


IMU 2

Huaishu Peng | UMD CS | Spring 2026



3-axis accelerometer + 3-axis gyroscope
Measures motion, orientation, and angular velocity

Accelerometer

Range: $\pm 2g$, $\pm 4g$, $\pm 8g$, $\pm 16g$

Gyroscope

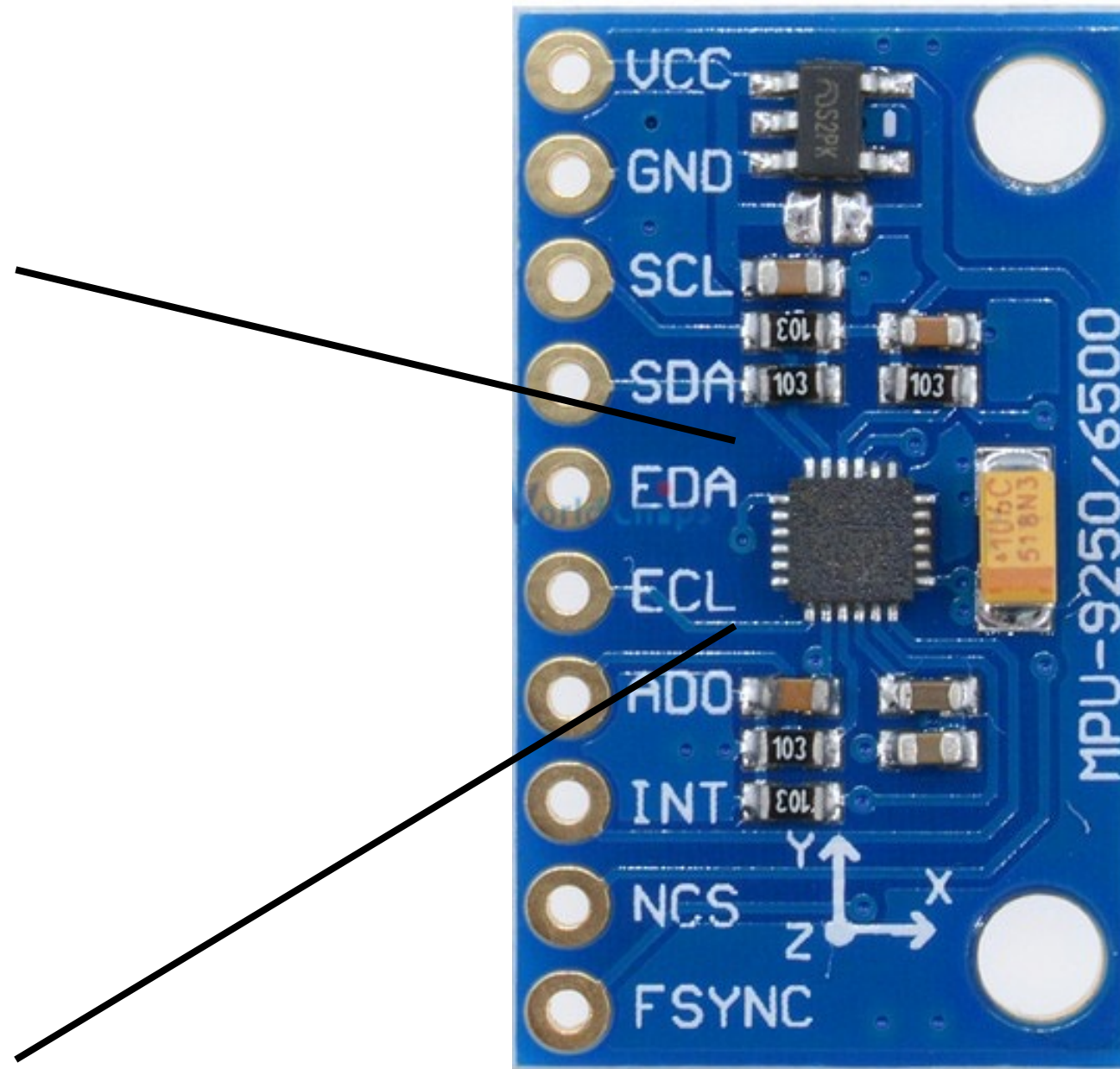
Range: ± 250 , ± 500 , ± 1000 , ± 2000 $^{\circ}/s$

SPI + I²C



Invensense MPU-6500

<https://datasheet.octopart.com/MPU-6500-InvenSense-datasheet-138896167.pdf>



I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

Address: 0b1101000
(0x68)



```
void loop() {  
  Wire.beginTransmission(address);  
  Wire.write(0x3B);  
  Wire.endTransmission();  
  
  Wire.requestFrom(address, 1);  
  ACCEL_XOUT_H = Wire.read();  
  
  Wire.beginTransmission(address);  
  Wire.write(0x3C);  
  Wire.endTransmission();  
  
  Wire.requestFrom(address, 1);  
  ACCEL_XOUT_L = Wire.read();  
  
  ACCEL_X_RAW = ACCEL_XOUT_H << 8 | ACCEL_XOUT_L;  
  
  gX = ACCEL_X_RAW / 16384.0;  
  
  //Plot data  
  Serial.print(">gX:");  
  Serial.println(gX);  
  delay(10);  
}
```

I²C library in Arduino - Wire library

Practice: Read temperature of the sensor in degrees C

I²C library in Arduino

Setup

Reading a register

Updating a register

4.18 Registers 65 and 66 – Temperature Measurement TEMP_OUT_H and TEMP_OUT_L

Type: Read Only

Register (Hex)	Register (Decimal)	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
41	65	TEMP_OUT[15:8]							
42	66	TEMP_OUT[7:0]							

Description:

These registers store the most recent temperature sensor measurement.

Temperature measurements are written to these registers at the Sample Rate as defined in Register 25.

These temperature measurement registers, along with the accelerometer measurement registers, gyroscope measurement registers, and external sensor data registers, are composed of two sets of registers: an internal register set and a user-facing read register set.

The data within the temperature sensor’s internal register set is always updated at the Sample Rate. Meanwhile, the user-facing read register set duplicates the internal register set’s data values whenever the serial interface is idle. This guarantees that a burst read of sensor registers will read measurements from the same sampling instant. Note that if burst reads are not used, the user is responsible for ensuring a set of single byte reads correspond to a single sampling instant by checking the Data Ready interrupt.

The scale factor and offset for the temperature sensor are found in the Electrical Specifications table (Section 6.4 of the MPU-6000/MPU-6050 Product Specification document).

The temperature in degrees C for a given register value may be computed as:

$$\text{Temperature in degrees C} = (\text{TEMP_OUT Register Value as a signed quantity})/340 + 36.53$$

Please note that the math in the above equation is in decimal.

I²C library in Arduino

Setup

Reading a register

Updating a register

```
#include <Arduino.h>
#include <Wire.h>

const uint8_t IMU_ADDR = 0x68;
const int sda = 21;
const int scl = 22;

void setup() {
  Serial.begin(115200);
  Wire.begin(sda, scl);

  delay(100);
}

void loop() {
  Wire.beginTransmission(IMU_ADDR);
  Wire.write(0x41);          // TEMP_OUT_H
  Wire.endTransmission();

  Wire.requestFrom(IMU_ADDR, 2);

  uint8_t highByte = Wire.read();
  uint8_t lowByte  = Wire.read();

  int16_t tempRaw = (highByte << 8) | lowByte;
  float tempC = (tempRaw / 340.0f) + 36.53f;

  Serial.print("Temp: ");
  Serial.print(tempC, 2);
  Serial.println(" C");

  delay(500);
}
```

Why might we want to change the measurement range?

3.2 Accelerometer Specifications

Typical Operating Circuit of section [4.2](#), VDD = 2.5V, VDDIO = 2.5V, T_A=25°C, unless otherwise noted.

PARAMETER	CONDITIONS	MIN	TYP	MAX	UNITS
Full-Scale Range	AFS_SEL=0		±2		<i>g</i>
	AFS_SEL=1		±4		<i>g</i>
	AFS_SEL=2		±8		<i>g</i>
	AFS_SEL=3		±16		<i>g</i>
ADC Word Length	Output in two's complement format		16		bits
Sensitivity Scale Factor	AFS_SEL=0		16,384		LSB/ <i>g</i>
	AFS_SEL=1		8,192		LSB/ <i>g</i>
	AFS_SEL=2		4,096		LSB/ <i>g</i>
	AFS_SEL=3		2,048		LSB/ <i>g</i>

With a ±16*g* range:

$$\frac{1g}{2,048 \text{ steps}} = 0.00048828g \text{ per step}$$

(0.49 mg per step)

With a ±2*g* range on the same ADC:

$$\frac{1g}{16,384 \text{ steps}} = 0.00006104g \text{ per step}$$

(0.061 mg per step)

I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

Let's try to read acceleration data **in the range of $\pm 16g$**
so that we can **detect strong sudden motions!**

I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

4.7 Register 28 – Accelerometer Configuration

Serial IF: R/W
Reset value: 0x00

BIT	NAME	FUNCTION
[7]	ax_st_en	X Accel self-test
[6]	ay_st_en	Y Accel self-test
[5]	az_st_en	Z Accel self-test
[4:3]	ACCEL_FS_SEL[1:0]	Accel Full Scale Select: ±2g (00), ±4g (01), ±8g (10), ±16g (11)
[2:0]	-	Reserved

```
Wire.beginTransmission(addr) //opens communication with the slave device with its addr
Wire.write(data) //prepares to send data to addr
Wire.write(data) //prepares to send data to addr
...
Wire.endTransmission() //sends the data and returns
```

I²C library in Arduino - Wire library

Setup

Reading a register

Updating a register

```
4
5
6 void setup() {
7   Serial.begin(9600);
8   Wire.begin();
9
10  Wire.beginTransmission(MPU);
11  Wire.write(0x1C); //Talk to the ACCEL_CONFIG Register
12  Wire.write(0b00011000); //Set the register bits for 16g scale
13  Wire.endTransmission();
14 }
```

4.7 Register 28 – Accelerometer Configuration

Serial IF: R/W
Reset value: 0x00

BIT	NAME	FUNCTION
[7]	ax_st_en	X Accel self-test
[6]	ay_st_en	Y Accel self-test
[5]	az_st_en	Z Accel self-test
[4:3]	ACCEL_FS_SEL[1:0]	Accel Full Scale Select: ±2g (00), ±4g (01), ±8g (10), ±16g (11)
[2:0]	-	Reserved

- Wire.beginTransmission(addr) //opens communication with the slave device with its addr
- Wire.write(data) //prepares to send data to addr
- Wire.write(data) //prepares to send data to addr
- ...
- Wire.endTransmission() //sends the data and returns

I²C library in Arduino - Wire library

Setup

Sensitivity Scale Factor	AFS_SEL=0		16,384		LSB/g
	AFS_SEL=1		8,192		LSB/g
	AFS_SEL=2		4,096		LSB/g
	AFS_SEL=3		2,048		LSB/g

Reading a register

```
gX = ACCEL_X_RAW / 2048.0;
```

Updating a register

```
Wire.beginTransmission(addr) //opens communication with the slave device with its addr
Wire.write(data) //prepares to send data to addr
Wire.write(data) //prepares to send data to addr
...
Wire.endTransmission() //sends the data and returns
```

Potential Issues with I2C Protocol?

SPI

SPI (Serial Peripheral Interface) is also a serial communication protocols used for connecting microcontrollers to peripheral devices.

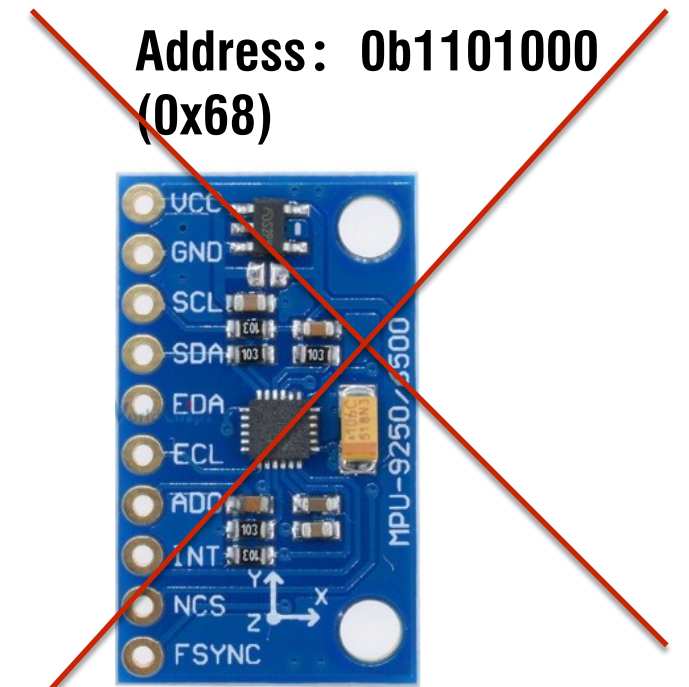
Wiring: Uses 4 main lines—

MISO (Master In, Slave Out)

MOSI (Master Out, Slave In)

SCLK (Serial Clock)

CS (Chip Select). Each peripheral device requires its own CS line



Benefit of using 4 wires (a more complex wiring?)

faster than I²C, supporting speeds up to several tens of MHz.

Full-duplex communication (data can be sent and received simultaneously), making it faster for bidirectional data transfer.

No device addressing, as each slave is selected by a dedicated CS pin.

I2C vs SPI

I2C:: fewer wires, easier to share, slower

SPI:: more wires, faster, simpler data transfer

Wiring: Uses 4 main lines—

MISO <-> AD0 <-> GPIO19 (SPI data output)

MOSI <-> SDA <-> GPIO23 (SPI data input)

SCLK <-> SCL <-> GPIO22

CS <-> NCS <-> GPIO5

```
#include <SPI.h>
```

```
//Initializes the ESP32 hardware SPI bus
```

```
SPI.begin(sck, miso, mosi, cs);
```

```
//clock speed->1MHz, bit order, and data mode
```

```
SPISettings imuSPI(1000000, MSBFIRST, SPI_MODE0);
```

```
SPI.beginTransaction(imuSPI);
```

```
digitalWrite(cs, LOW);
```

```
SPI.transfer(0x3B | 0x80);
```

```
ACCEL_XOUT_H = SPI.transfer(0x00);
```

```
ACCEL_XOUT_L = SPI.transfer(0x00);
```

```
digitalWrite(cs, HIGH);
```

```
SPI.endTransaction();
```

0x3B = 00111011

0x80 = 10000000

10111011 = 0xBB

For many SPI sensors, that
top bit is used as a read
flag: 0 = write 1 = read


```
SPI.beginTransaction(imuSPI);  
digitalWrite(cs, LOW);
```

```
SPI.transfer(0x3B | 0x80);  
ACCEL_XOUT_H = SPI.transfer(0x00);  
ACCEL_XOUT_L = SPI.transfer(0x00);
```

```
digitalWrite(cs, HIGH);  
SPI.endTransaction();
```

SPI is full-duplex: every time you send a byte, you also receive a byte. So to receive the data, we need to send something, like dummy byte

```
SPISettings imuSPI(1000000, MSBFIRST, SPI_MODE0);

void setup() {
  Serial.begin(9600);

  pinMode(cs, OUTPUT);
  digitalWrite(cs, HIGH);

  SPI.begin(sck, miso, mosi, cs);

  delay(100);
}

void loop() {

  SPI.beginTransaction(imuSPI);
  digitalWrite(cs, LOW);

  SPI.transfer(0x3B | 0x80); // read
  ACCEL_XOUT_H = SPI.transfer(0x00);
  ACCEL_XOUT_L = SPI.transfer(0x00);

  digitalWrite(cs, HIGH);
  SPI.endTransaction();

  ACCEL_X_RAW = (ACCEL_XOUT_H << 8) | ACCEL_XOUT_L;
  gX = ACCEL_X_RAW / 16384.0;

  Serial.print(">gX:");
  Serial.print(gX);
  Serial.println(" g");

  delay(10);
}
```

I²C library in Arduino - Wire library

Understanding Gyro Data

BIT	NAME	FUNCTION
[7:0]	D[7:0]	Low byte of the X-Axis gyroscope output GYRO_XOUT = Gyro_Sensitivity * X_angular_rate Nominal FS_SEL = 0 Conditions Gyro_Sensitivity = 131 LSB/(°/s)

degree per second

I²C library in Arduino

Understanding Gyro Data

```
gyroX_Per_S = GYRO_X_RAW/131.0;  
gyroY_Per_S = GYRO_Y_RAW/131.0;  
gyroZ_Per_S = GYRO_Z_RAW/131.0;
```

angular velocity

```
currentTime = millis();  
elapsedTime = (currentTime - previousTime) / 1000;
```

```
gyroAngleX = gyroAngleX + gyroX_Per_S * elapsedTime;  
gyroAngleY = gyroAngleY + gyroY_Per_S * elapsedTime;  
gyroAngleZ = gyroAngleZ + gyroZ_Per_S * elapsedTime;
```

I²C library in Arduino

Dead Reckoning

- Drifting over time because errors accumulated and built upon previous measurements -> data won't be accurate
- Still, we can reduce the error with a simple calibration process
- For example, you can record 10s of raw x y z gyro data to find the average offset -> which you can plug into your final output
- Offset varies based on your device

```
GYRO_X_RAW -= gyroXOffset;  
GYRO_Y_RAW -= gyroYOffset;  
GYRO_Z_RAW -= gyroZOffset;  
  
gXrate = GYRO_X_RAW / 131.0;  
gYrate = GYRO_Y_RAW / 131.0;  
gZrate = GYRO_Z_RAW / 131.0;
```

